



# C

# Programming

Distributed by :

*Script Web Solution Pvt. Ltd.*

## About Programming

Programming is nothing more than instructing the computer to perform certain task. It is the process of writing different programs or instructions in different High Level and Low level Languages (the languages that are used for programming in computer) using certain rigid rules known as SYNTAX. Syntax is a rule that is used for writing different source codes.

Each and every task performed by a computer is guided by a set of programs or software. Generally Low level Languages like Machine Language and Assembly Languages are used for developing system software while the High Level Languages like QBASIC, Visual Basic, 'C', Java etc are used for developing Application Software.

## C Language

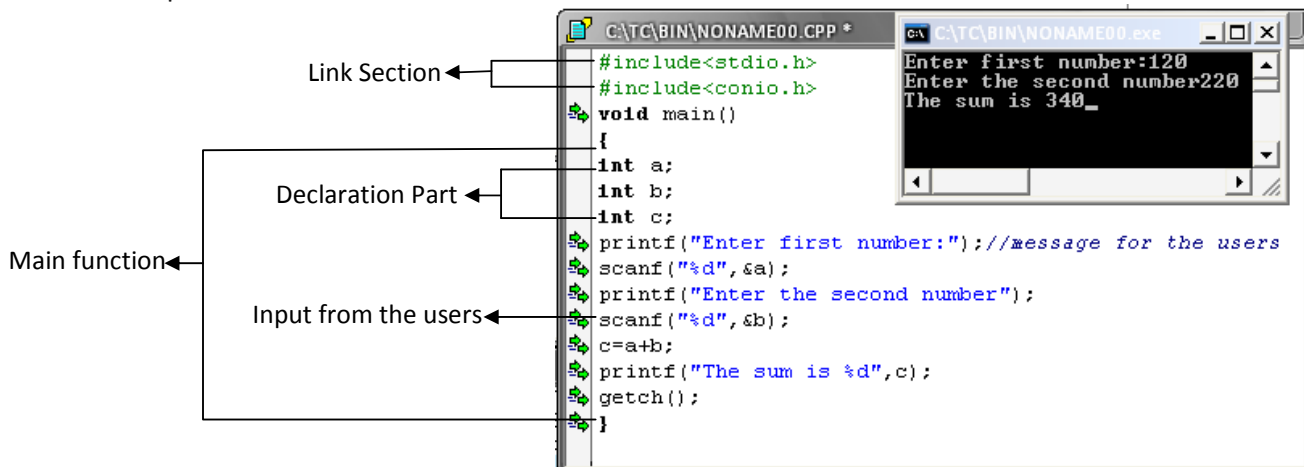
C is a very powerful computer programming language that combines all the features of High Level Language with the capabilities of Assembly Language. Thus, it is generally referred as a Middle Level Language. In this respect we can develop both Application Software as well as System Software with the help of C language.

The UNIX operating system is entirely coded in C. Different applications so far are also coded in C language.

### Features

- Highly portable
- Features of Both high level and Low Level Languages
- Very fast processing
- Programs being divided into different small modules make the program readable and easy for debugging

Let's consider a sample C program that asks two numbers from the users and displays their sum as output:



The link section provides instructions for the compiler to link certain files from the system library that contains the functions used in our program.

The `main()` function is one of the user defined functions which is the main executable part of our program. Every C program must contain a `main()` function but not more than that.

The opening braces '{' indicates the starting of execution while the execution ends with the closing brace '}'.

Different variables that are used in our program are declared in the Variable Declaration part. Each and every variables used in our program must be declared and assigned certain data types before use.

The `printf()` function is used to display different message or output in the monitor while the `scanf()` function, to ask certain values from the users through keyboard.

Each statements in C ends with a semicolon (;).

Each opening brace must have corresponding closing brace.

### Keywords and identifiers

Each and every words in C are either identifiers or keywords.

### Keywords

Keywords are the special words reserved by the compiler to perform certain functions. There are altogether 32 keywords in C. Some of them are:

**int float if else for while switch do etc.**

Every keyword must be written in lowercase.

### Identifiers

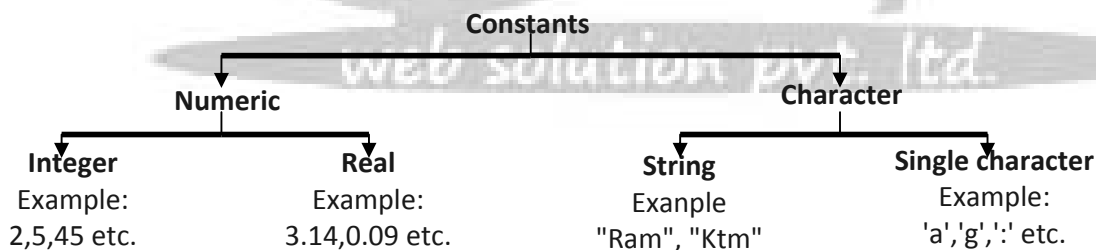
Identifiers are the names given to certain functions, variables and arrays. Eg: name, num1, \_aa, f\_name etc.

#### Rules for writing keywords:

- Identifiers may compose of letters and digits but the first character must always be a letter.
- It can contain upto 31 characters, but 8 characters is much preferable.
- Keywords cannot be used as Identifiers.
- Identifiers are generally written in lower case. However uppercase is also permitted.

### Constants

Constants are the values that don't change during the execution of the whole program. The following figure describes the types of constants.



### Variables

Variables are the identifiers that are used to store certain values in our program for temporary and permanent use. Unlike constants the value of the variable may change several times during the execution of certain program. Eg: Num1, sum etc

### Data types in C

C supports a variety of data types that supports different types of data. Certain data types are assigned to the variables so as store particular types of values. Primarily, there are four data types in C, namely,

- |                                      |        |                                                 |
|--------------------------------------|--------|-------------------------------------------------|
| i. Integer                           | int    | supports the integer values                     |
| ii. Character                        | char   | supports string and characters                  |
| iii. Single precision Floating point | float  | supports single precision floating point values |
| iv. Double precision floating Point  | double | supports double precision floating point values |

Different **qualifiers** may be added to the data types in order to change the storage property of the data types. The following are some of the **qualifiers** used in C.

**Signed Unsigned Long Short**

### Declaration of variables

All the variables used in our program must be declared before use.

Syntax:

**[data type] [Variable name];**

OR

**[Data type] v1, v2, v3,....., vn;**

Example:

int number; Char name[10]; int a,b,c; etc.

### Assigning values to variables

Syntax:

**[Variable name]= [value] or (expression)**

Example:

A=10; Num=52.5; c=a+b; etc.

Values can also be assigned to the variables in the period of declaration. Like `int a=5; etc.`

### Header files in C

Header files are the files contained in the **C** library that contains certain set of pre defined functions used in our program. The following are some of the commonly used header files.

**stdio.h** : It contains the standard input output functions like `printf()`, `scanf()`, `gets()`, `puts()` etc.

**conio.h**: It contains some of the functions like `getch()`, `clrscr()` etc.

**math.h** : It contains the mathematical functions like `pow()`, `sqrt()`, `abs()`, `fmod()` etc.

**String.h**: It contains the string manipulating functions like `strlen()`, `strcpy()`, `strcmp()`, `strrev()` etc.

**ctype.h** : It contains some validating function like `isdigit()`, `isalnum()`, `isalpha()` and some value converting functions like `toupper()`, `tolower()` etc.

### Displaying output in the monitor

We can display output in the monitor using the following functions:

**Printf()**: The `printf()` function is the most commonly used output displaying function by which we can display the output in our required format.

Syntax:

**`printf("control string", variable names);`**

Every text in between the double quotes(control string) is printed as it is except the format specifier and the escape sequence characters.

#### Format specifier:

| Character | meaning                                            |
|-----------|----------------------------------------------------|
| %d        | for calling integer type of value                  |
| %s        | for calling string type of value                   |
| %c        | for calling single character values                |
| %f        | for calling single precision floating point values |
| %lf       | for calling double precision floating point values |
| %u        | for calling unsigned integer values                |

#### Escape sequence characters:

| Character | meaning                                                      |
|-----------|--------------------------------------------------------------|
| \t        | horizontal tab(leave some space between two strings or text) |
| \n        | new line                                                     |
| \f        | form feed                                                    |
| \a        | audible alert                                                |
| \v        | vertical tab                                                 |

Example:

`printf("This is my first program");`

`printf("My name is Charles\nI live in the UK");`

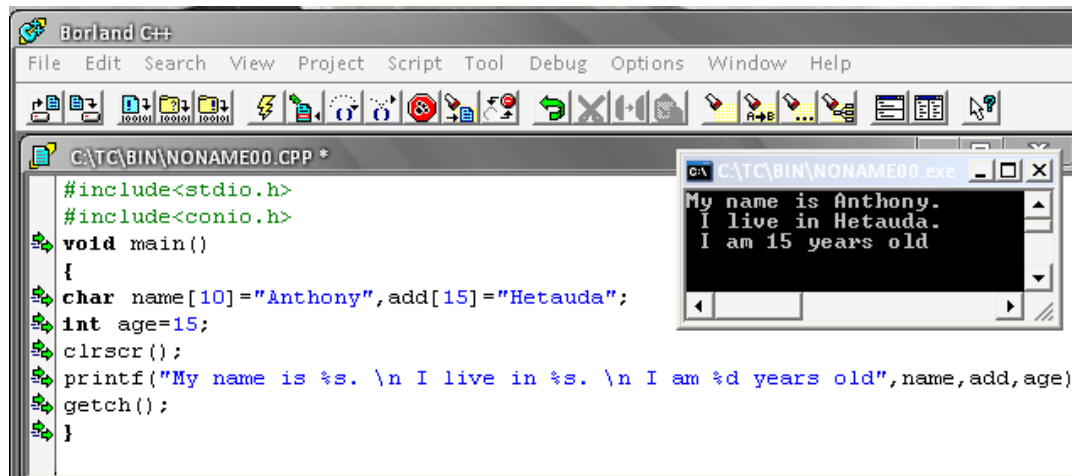


Fig: sample C program to print text.

## String.h

It contains the string manipulating functions like `strlen()`, `strcpy()`, `strcmp()`, `strrev()` etc.

**puts()**

The puts function can be used to display a large text in the monitor. The puts function is used only for the character data types and the strings.

Syntax:

Puts(variable);

OR

```
Puts("string");
```

Example:

Puts(a);                      Puts("Enter the number:"); etc.

## Putchar()

The `putchar()` function is also used to display the text on the monitor but unlike `puts()`, we can display only single character using this function. This is the property as well as the limitation of the `putchar()` function.

Syntax:

```
putchar(variable name); OR putchar('character');
```

Example:

```
Putchar(a);           putchar(code);           putchar('r');
```

## Accepting Input from the users

The following function is commonly used to accept input from the users through keyboard.

## Scanf()

It is the most commonly used function for accepting input from the users.

Syntax:

```
Scanf("control string", variables);
```

The control string may contain any of the format specifier as per our requirement. The first format specifier is used for the first variable and the rest for the corresponding ones. The scanf() function terminates as soon as it encounters the blank space in case of string.

Example:

```
scanf("%d",&a); scanf("%d %s",&age,name);
```

**Gets()**

This function is used to read a string of words(text or sentence) from the keyboard and store in the given variable.

Syntax:

Gets(variable name);

Example:

Gets(sentence);                      gets(message);

**Getch()**

It is used to read a single character from the keyboard and store in a variable, if necessary.

Syntax:

Getch();                      variable name=getch() ;

**Operators in C**

C supports a rich set of operators. An **operator** is a sign or symbol that is used to perform certain type of mathematical and logical manipulations. It is generally used to combine two or more operands and perform the operation accordingly.

The following are the commonly used operators:

**Arithmetic Operators**

The arithmetic operators include the basic operators used for performing certain mathematical operations like addition, multiplication etc.

| <u>Task</u>     | <u>operators</u>                        |
|-----------------|-----------------------------------------|
| Addition        | +                                       |
| Subtraction     | -                                       |
| Multiplication  | *                                       |
| Division        | /                                       |
| Modulo division | % (return the remainder after division) |

**Relational operators**

The relational operators are used to compare two different quantities and produce a result. These operators are often used in combination with the for, while loops, if conditions etc.

| <u>Operators</u> | <u>Meaning</u>           |
|------------------|--------------------------|
| >                | Greater than             |
| <                | Less than                |
| ≥                | greater than or equal to |
| ≤                | Less than or equal to    |
| ==               | is equal to              |
| !=               | not equal to             |

**Logical operator**

The logical operators are used to combine two or more relational expressions and produce output accordingly.

| <u>Operators</u> | <u>Meaning</u> |
|------------------|----------------|
| &&               | Logical AND    |
|                  | Logical OR     |
| !                | Logical NOT    |

**Assignment operators**

These operators are used to assign values to the variables. The equal to(=) sign is the only operator used for assigning values to the variables. However some shorthand operators can also be used in order to perform certain calculations and assign values to the variables.

Some shorthand operators

`a+=b` is equivalent to `a=a+b` and similarly `--` `*=` `\=` `%=`

**Unary operators**

The unary operators work in a single operand only. The unary operators `++` and `--` are used to either increase or decrease the value of a variable by unity(one). Plus(+) and Minus(−) are another types of unary operators used with single operand.

Example:

`a++;` is equivalent to `a=a+1;` similar is `a--` `--a` `++a` etc.

**Conditional Operator**

The conditional operators are the new important feature of C language which is used to check certain relational expression and execute the true statement if the condition is true and display the false statement if the condition is false.

Syntax:

`[Condition] ? [true statement] : [false statement];`

Example:

`a>b?printf("a is greater"):printf("b is greater");`

**Bitwise operator**

The bitwise operators are used for having bit level computations of different values.

| <u>Operators</u>      | <u>Meaning</u>       |
|-----------------------|----------------------|
| <code>&amp;</code>    | bitwise AND          |
| <code> </code>        | bitwise OR           |
| <code>^</code>        | bitwise exclusive OR |
| <code>&gt;&gt;</code> | shift cells right    |
| <code>&lt;&lt;</code> | shift cells left     |
| <code>~</code>        | One's complement     |

**Comma operator**

The comma operator is usually used to combine two or more similar types of expressions.

Example:

`For(i=1,a=5;i<=5;i++,a=a*5)`  
`t=a, a=b, b=t;`

*Some other operators like dot operator, sizeof operator etc. are also used for some special purposes which will be discussed in following chapters.*

**Exercises:**

- ❖ WAP to print any message on the screen.
- ❖ WAP to ask any number or string from the users and display it on the monitor.
- ❖ WAP to find the sum of two numbers entered by the users.
- ❖ Write the C code for the following expression

$$a) \ x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$b) \ \sqrt[4]{a^2 + b^2}$$

$$c) \ C = \frac{F - 32}{8}$$

- ❖ WAP to find the area of a circle if radius is given. Find the circumference also.
- ❖ WAP to solve the quadratic equation of the form  $ax^2 + bx + c$  if a, b and c is supplied by the users.(Formula above).
- ❖ WAP to convert months into days.(1 month=30 days)
- ❖ WAP to convert temperature in Fahrenheit to Celsius if Fahrenheit is given.(formula above)
- ❖ Write the output of the following program:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,a,age=25;
    float k=3.14167;
    a=65;
    char name[10]="Ramesh",character=f;
    clrscr();
    printf("My name is %s.\nI am %d years old.\nAnd I live in Kathmandu.\n",name,age);
    printf("%2.2f",k);
    printf("%3s",name);
    printf("%c",a);
    printf("%d",character);
    getch();
}
```

- ❖ WAP to convert days to months if no. of days are given.
- ❖ WAP to find the sum of the digits of any four digit number without using loops.
- ❖ WAP to reverse any four digit number.
- ❖ WAP to display the Simple interest if principal, rate and time is supplied by the user.

# Decision making and Branching

Different decision making statements can be used in **C** in order to test different conditions and perform the execution accordingly. The decision making statements are also known as branching statements. Branching is the process of transferring the control of a program to different sections according as the given condition is true or false.

The if....else and Switch.....Case statements are often used for branching.

## If statement

The **if** statement is a powerful and efficient decision making(conditional) statement which is used to test a conditional expression and execute the statements immediately following it if the given condition is true and not if the condition is false. The control terminates out of the **if** statement if the given condition is false.

Syntax:

```
if(condition)
{
    statement block;
}
```

Example:

```
if(age<15)
{
    printf("You are a Child");
}
```

The **if.....else** statement can be used if there are two different values for a single condition (one true statement and the other false)

Syntax:

```
if(condition)
{
    True statement block;
}
else
{
    False statement block
}
```

Example:

```
if(a>b)
{
    printf("%d is greater",a);
}
else
{
    printf("%d is greater",b);
}
```

## Nested if statement

The nested if statement is used if the program passes through a series of condition. The If statement can be written (nested) within another if statement also which is known as **nested if** statement.

Syntax:

```
if(condition1)
{
    if(condition 2)
    {
        True statement block 2;
    }
    else
    {
        False statement block 2;
    }
}
else
{
    False statement block 1;
}
```

Example:

```
if(gender=="female")
{
    if(age>25)
    {
        printf("Congratulations You can apply!")
    }
    else
    {
        printf("sorry, you can't apply")
    }
}
else
{
    printf("sorry you can't apply");
}
```

**If.....else if.....else statement**

The **if....else if** statement is used if a single condition contains a multiple number of alternatives.

Syntax:

```

if(condition 1)
{
Statement block 1;
}
else if(condition 2)
{
Statement block 2;
}
:
:
else if(condition n)
{
Statement block n;
}
else
{
Default statement;
}

```

```

if(percent>=60)
{
printf("first");
}
else if(percent>=45)
{
printf("second");
}
else if(percent>=35)
{
printf("Third");
}
else
{
printf("fail");
}

```

**The goto statement**

The goto statement is used to transfer the control of the program from one point to another fixed point given by label. Sometimes we can also perform looping by using this statement. However, the main function of goto statement is not looping but jumping from one part of the program to other, whether it be to or fro of a loop or a simple program.

Syntax:

```

:
:
Label name:
:
goto [label name];
:

```

The label name may contain any number or text but it must end with a colon(:). It can appear in any part of the program as per our requirement. The goto statement can also be placed in any part of the program.

**Switch.....case statement**

The switch case statement is another important branching statement that divides a program into multiple cases by assigning different case values. Then the switch() statement is used to switch(transfer the control) to the selected case. Each section are the alternatives for the given condition. It is very important for coding menu base program.

Syntax:

```

switch (variable name)
{
    case [case value 1]:
        statement block 1;
        break;
    case [case value 2]:
        statement block 2;
        break;
    :
    :
    case [case value n]:
        statement block n;
        default:
        default statement;
}

```

```

switch(day)
{
case 1:
puts("Sunday");
break;
case 2:
puts("Monday");
break;
:
case 7:
puts("Saturday");
break;
default:
perror("Wrong entry");
}

```

**The break statement**

The break statement is used to terminate and transfer the control out of certain program (any looping statements or switch case statement) when the program meets certain condition. The program skips all other statements within the loop or the switch statement as soon as it encounters the break statement.

Syntax:

```
break;
```

```

C:\ABC5\BIN\BKG3.C *
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    printf("A program for Calculations\n\n\n");
    int num1,num2;
    printf("enter first number:");scanf("%d",&num1);
    printf("enter second number:");scanf("%d",&num2);
    printf("\n\nWhat would You like to do:\n");
    printf("1.Add\n2.Subtract\n3.Multiply\n");
    printf("Enter your choice here:");
    int choice;
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:
            printf("The sum is %d",num1+num2);
            break;
        case 2:
            printf("The difference is %d",num1-num2);
            break;
        case 3:
            printf("The product is %d",num1*num2);
            break;
        default:
            printf("Wrong entry");
    }
    getch();
}

```

Execution Output:

```

A program for Calculations
enter first number:10
enter second number:42

What would You like to do:
1.Add
2.Subtract
3.Multiply
Enter your choice here:3
The product is 420_

```

Fig: A menu base program to calculate two different numbers as provided by the users to demonstrate the use of switch statement

**Exercises:**

- WAP to check whether the number entered by the user is odd or even.
- WAP to check whether a number entered by the user is positive or negative.
- WAP to check whether the number entered by the user is divisible by 2 and 5 or not.
- WAP to find the greatest number among two numbers.
- WAP to ask three numbers from the user and display:
  - Greatest number
  - Middle number
  - Smallest number
- WAP to check whether a three digit number entered by the user is Armstrong or not.
- WAP to check whether a character entered by the user is vowel or consonant.
- WAP to calculate the division if the percentage is given.
- WAP to print a perfect mark sheet of a student if the Marks in all subjects are supplied by the users.
- WAP to print a salary sheet for a employee with Bonus, Income tax and Net Salary if the salary is given.

Conditions:

**Bonus:** if salary is greater than 10000 then 10% of the salary , if salary is greater than 5000 and less than 10000 then 50% of the salary and if not just 3% of the salary.

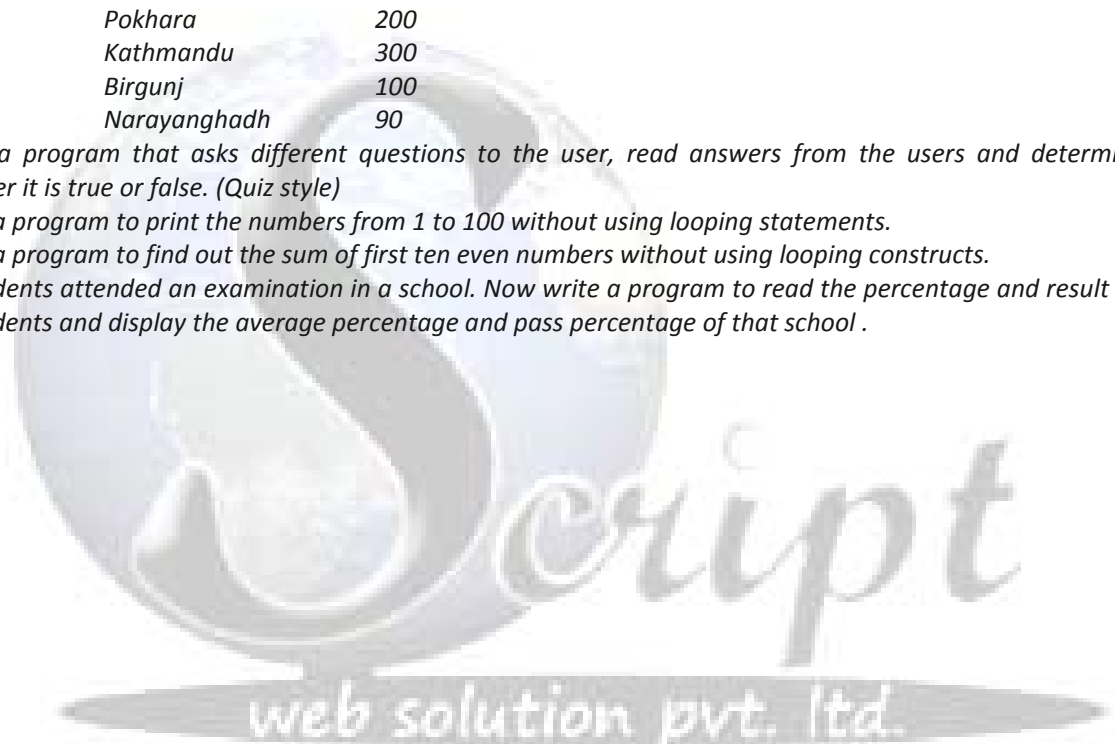
**Income Tax:** if salary is greater than 15000 then 13% else no tax.

- k) Write a program to print the days of a week according as user inputs 1 to 7. eg(1->Sunday,2 ->Monday and so on) [And similarly write a program for months]
- l) Write a menu base program using switch.....case statement that asks the distination and the no. of passangers and display the total bill amount according to the destination selected.

Price list:

|              |     |
|--------------|-----|
| Pokhara      | 200 |
| Kathmandu    | 300 |
| Birgunj      | 100 |
| Narayanghadh | 90  |

- m) Write a program that asks different questions to the user, read answers from the users and determine whether it is true or false. (Quiz style)
- n) Write a program to print the numbers from 1 to 100 without using looping statements.
- o) Write a program to find out the sum of first ten even numbers without using looping constructs.
- p) 50 students attended an examination in a school. Now write a program to read the percentage and result of 50 students and display the average percentage and pass percentage of that school .



## Looping

Looping statement is another important type of control flow statement that is used to execute certain set of statements for multiple number of times. **Looping** as its name implies, is the process of executing a certain set of statements repeatedly for multiple number of times until a given condition is true. **Looping** is also known as **Iteration**.

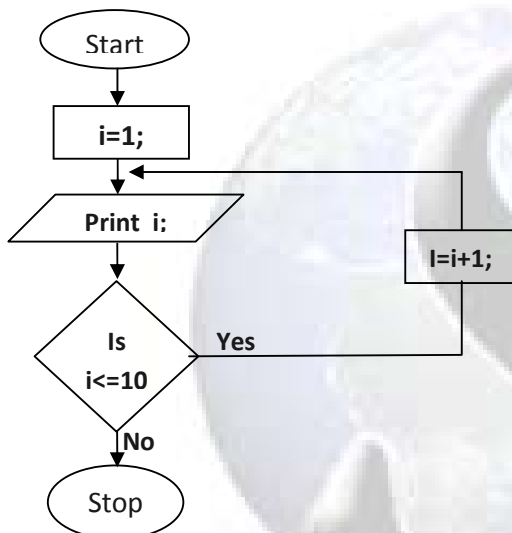


Fig: A flowchart of a program showing a simple loop.

```

#include<conio.h>
#include<stdio.h>

main()
{
    for(int i=1;i<=10;i++)
    {
        printf("%d\n",i);
    }
    getch();
}
  
```

Fig: Source code with output for the flowchart aside.

There are two types of looping statements according to their nature:

- i) **Finite loop:** The loop that ends at a certain point.
- ii) **Infinite Loop:** The Loop that doesn't end.

```

#include<stdio.h>
#include<conio.h>

main()
{
    int a=2;
    while(a!=0)
    {
        printf("%d\t",a);
        a=a+2;
    }
    getch();
}
  
```

Fig: An example of the infinite loop that stops only when a=0. But the value of 'a' never become Zero during the execution of the whole program.

The following statements are used in most of the programming languages for **Looping**:

**i) The While loop:**

It is a powerful and efficient looping statement that checks for a certain condition and then executes the given set of statements within the scope of loop until the given condition is true. The while loop is known as **entry control loop** because it checks the condition before executing the statements. It executes the statements within the loop only when the given condition is true. i.e It doesn't execute the statements even for a single time if the condition is false. The program is terminated as soon as the condition is evaluated false.

Syntax:

```
While(condition)
{
    Statement block;
}
```

Example:

```
char x= 'a';
while(x<= 'z')
{
    printf("%c => %d",x,x);
    x++;
}
```

**ii) The Do.....While loop:**

It is another looping statement which is almost similar to the while loop but unlike while loop the statements inside the do.....while loop is executed for at least one time even if the given condition is false. The do....while loop also contains a condition as the while loop but it executes the statements first and then checks the condition so it is known as **exit control loop**. The do....while statement ends with a semicolon(;).

Syntax:

```
do
{
    statement block;
}while(condition);
```

Example:

```
a=1;
do
{
    c=c+a;
}while(a<=10);
printf("The sum is %d",c);
```

**iii) The for loop:**

For loop is the most commonly used looping statement that is used for repeating the execution of a set of statements for multiple number of times until the given condition is true. It is also a entry control loop that checks the condition before the execution of statements.

Syntax:

```
for(initialization ; condition ; increment/decrement)
{
    Statement block;
}
```

The for loop contains three sections which is given by initialization, condition and Increment\decrement.

Example:

```
for(i=1;i<=100;i++)
{
    printf("%d\t",i);
}
```

```
for(a=50,a>=1,a=a-2)
{
    printf("%d\t",a);
    sum=sum+a;
}
printf("sum= %d",sum);
```

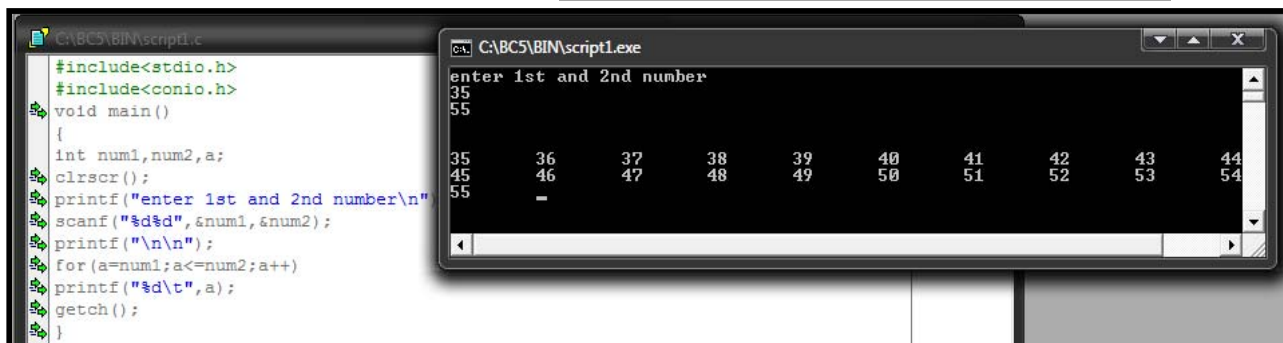
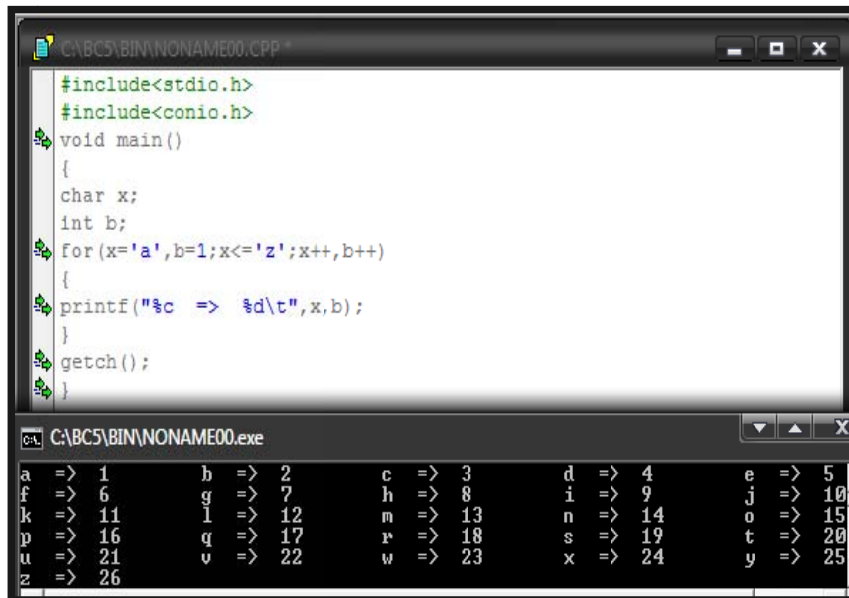


Fig: A program to print the numbers lying between two different numbers using for loop

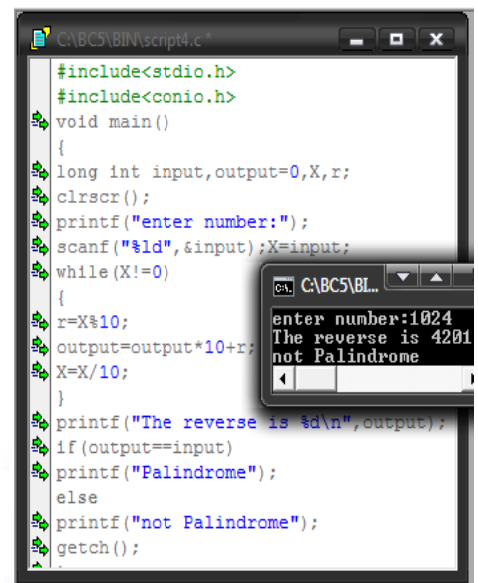


```
#include<stdio.h>
#include<conio.h>
void main()
{
    char x;
    int b;
    For(x='a',b=1;x<='z';x++,b++)
    {
        printf("%c => %d\t",x,b);
    }
    getch();
}
```

C:\BC5\BIN\NONAME00.exe

|         |         |         |         |         |
|---------|---------|---------|---------|---------|
| a => 1  | b => 2  | c => 3  | d => 4  | e => 5  |
| f => 6  | g => 7  | h => 8  | i => 9  | j => 10 |
| k => 11 | l => 12 | m => 13 | n => 14 | o => 15 |
| p => 16 | q => 17 | r => 18 | s => 19 | t => 20 |
| u => 21 | v => 22 | w => 23 | x => 24 | y => 25 |
| z => 26 |         |         |         |         |

Fig: A program that prints the alphabets from a-z and enumerates it.

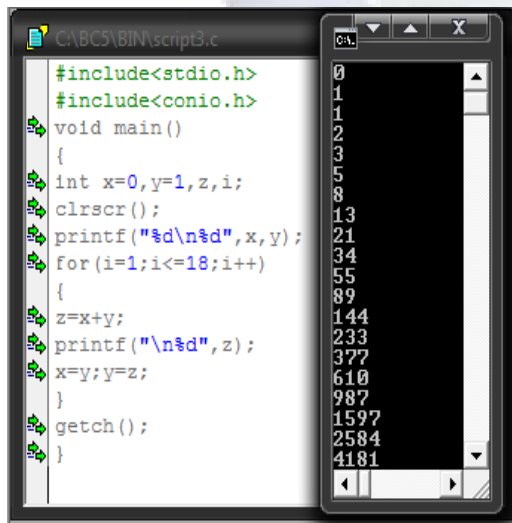


```
#include<stdio.h>
#include<conio.h>
void main()
{
    long int input,output=0,X,r;
    clrscr();
    printf("enter number:");
    scanf("%ld",&input);X=input;
    while(X!=0)
    {
        r=X%10;
        output=output*10+r;
        X=X/10;
    }
    printf("The reverse is %d\n",output);
    if (output==input)
    printf("Palindrome");
    else
    printf("not Palindrome");
    getch();
}
```

C:\BC5\BIN\script4.exe

enter number:1024  
The reverse is 4201  
not Palindrome

Fig: A program to reverse a multiple digit number and check for Palindrome.

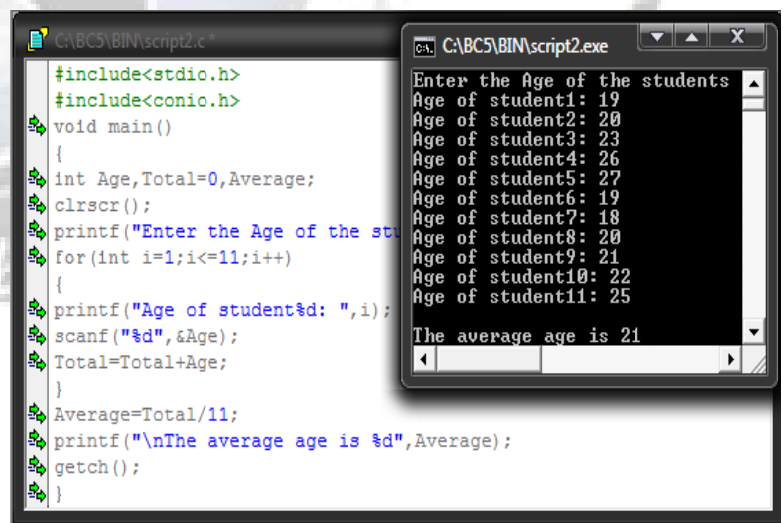


```
#include<stdio.h>
#include<conio.h>
void main()
{
    int x=0,y=1,z,i;
    clrscr();
    printf("%d\n%d",x,y);
    for (i=1;i<=18;i++)
    {
        z=x+y;
        printf("\n%d",z);
        x=y;y=z;
    }
    getch();
}
```

C:\BC5\BIN\script3.exe

0  
1  
1  
2  
3  
5  
8  
13  
21  
34  
55  
89  
144  
233  
377  
610  
987  
1597  
2584  
4181

Fig: A program to print the first 11 terms of the Fibonacci series.



```
#include<stdio.h>
#include<conio.h>
void main()
{
    int Age,Total=0,Average;
    clrscr();
    printf("Enter the Age of the students");
    for (int i=1;i<=11;i++)
    {
        printf("Age of student%d: ",i);
        scanf("%d",&Age);
        Total=Total+Age;
    }
    Average=Total/11;
    printf("\nThe average age is %d",Average);
    getch();
}
```

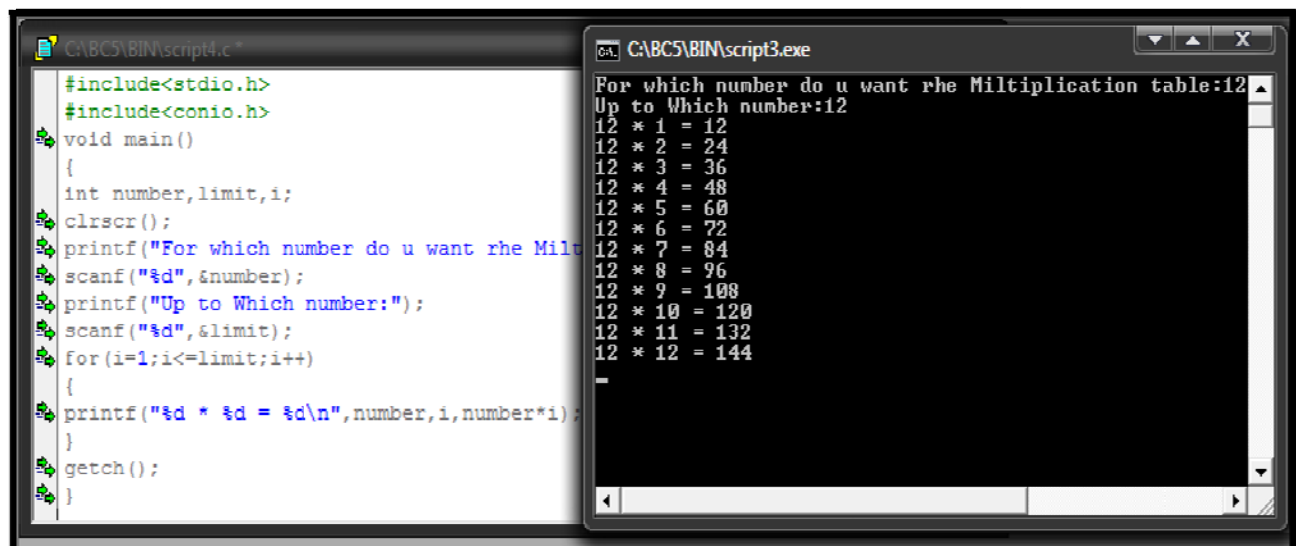
C:\BC5\BIN\script2.exe

Enter the Age of the students

Age of student1: 19  
Age of student2: 20  
Age of student3: 23  
Age of student4: 26  
Age of student5: 27  
Age of student6: 19  
Age of student7: 18  
Age of student8: 20  
Age of student9: 21  
Age of student10: 22  
Age of student11: 25

The average age is 21

Fig: A program to print the average age of 11 students if ages are supplied by the user.



```
#include<stdio.h>
#include<conio.h>
void main()
{
    int number,limit,i;
    clrscr();
    printf("For which number do u want rhe Miltiplication table:");
    scanf("%d",&number);
    printf("Up to Which number:");
    scanf("%d",&limit);
    for (i=1;i<=limit;i++)
    {
        printf("%d * %d = %d\n",number,i,number*i);
    }
    getch();
}
```

C:\BC5\BIN\script4.exe

For which number do u want rhe Miltiplication table:12  
Up to Which number:12

12 \* 1 = 12  
12 \* 2 = 24  
12 \* 3 = 36  
12 \* 4 = 48  
12 \* 5 = 60  
12 \* 6 = 72  
12 \* 7 = 84  
12 \* 8 = 96  
12 \* 9 = 108  
12 \* 10 = 120  
12 \* 11 = 132  
12 \* 12 = 144

Fig: A program to print the Multiplication table of the number entered by the user up to the given limit.

Sometimes a loop is inserted within another loop in order to run the inner loop for multiple numbers of times. Such loops nested within another loop are known as Nested loop. All three looping statements discussed above can be nested. One type of loop can be nested within another type also. For example a for loop can contain a while loop inside it or vice versa.

Syntax for Nested For:

```
for(.....)
{
    for(.....)
    {
        statement block;
    }
}
```

Syntax for Nested While:

```
while(condition)
{
    while(condition)
    {
        statement block;
    }
}
```

Syntax for Nested While:

```
do
{
    do
    {
        statement block;
    } while(condition)
} while(condition)
```

Example:

Write a program to print the multiplication table of the numbers from 1 to 10.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b;
    printf("A program to print the multiplication table from 1 to 10\n");
    for(a=1;a<=10;a++)
    {
        for(b=1;b<=10;b++)
        {
            printf("%d\t",b*a);
        }
        printf("\n");
    }
    getch();
}
```

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  |
|----|----|----|----|----|----|----|----|----|-----|
| 2  | 4  | 6  | 8  | 10 | 12 | 14 | 16 | 18 | 20  |
| 3  | 6  | 9  | 12 | 15 | 18 | 21 | 24 | 27 | 30  |
| 4  | 8  | 12 | 16 | 20 | 24 | 28 | 32 | 36 | 40  |
| 5  | 10 | 15 | 20 | 25 | 30 | 35 | 40 | 45 | 50  |
| 6  | 12 | 18 | 24 | 30 | 36 | 42 | 48 | 54 | 60  |
| 7  | 14 | 21 | 28 | 35 | 42 | 49 | 56 | 63 | 70  |
| 8  | 16 | 24 | 32 | 40 | 48 | 56 | 64 | 72 | 80  |
| 9  | 18 | 27 | 36 | 45 | 54 | 63 | 72 | 81 | 90  |
| 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b;
    for(a=1;a<=9;a++)
    {
        for(b=9;b>a;b=b-1)
        {
            printf("%d",b);
        }
        printf("\n");
    }
    getch();
}
```

98765432  
9876543  
987654  
98765  
9876  
987  
98  
9

```
void main()
{
    int a,b,c;
    clrscr();
    for(a=1;a<=7;a++)
    {
        for(b=1;b<=a;b++)
        {
            printf("*");
        }
        printf("\n");
    }
    for(a=1;a<=7;a++)
    {
        for(b=1;b<=a;b++)
        {
            printf("*");
        }
        printf("\n");
    }
    printf("\n");
    printf("||\n||\n||\n||\nFlag of NEPAL");
}
```

Flag of NEPAL

Fig: A demonstration of Nested for loop.

## Exercises:

1. What is the difference between While and DO...while loop?
2. Write a program using while loop to display the numbers from 1 to 10.
3. Write a program to display the odd numbers with their sum from 1 to ten.
4. Write a program to display the first ten even numbers.
5. Write a program to display the sum of numbers between two numbers entered by the users.
6. Write the output of the following looping statement:
 

```

      :
      for(i=1;i<=5;i++)
      {
      printf("Ram %d",i);
      }
      printf("%d",i);
      :
      
```
7. Write a program to print the **Fibonacci series** up to ten terms.
8. Write a program to count the number of digits of a integer number entered by the user.
9. Write a program to check whether a number entered by user is **Armstrong** or not. Such numbers are said to be **Armstrong** whose sum of cube of the digits is equal to that number.
10. Write a program to print the alphabets from A-Z and a-z using a single loop.
11. Write a program to ask ten numbers from the user and print only the **odd** ones.
12. Write a program to print the multiplication table of the number entered by the user.
13. Write a program to print the **Factorial** of the number given by the user.
14. Write a program to find out the greatest numbers among ten numbers entered by the user.
15. Write a program to read the age of 100 people and display the number of people between 30-50 age group.
16. Write a program to find out the sum of the the following series...  
 $1+x+x^2+x^3+...+x^n$  where x and n are supplied by the user.
17. Write a program to determine whether a number entered by the user is **prime** or **not**.
18. Write a program to count the number of **odd composite** numbers between 10 and 100.
19. Write a program to read a positive integer number and display its equivalent binary number.

**Nested Loop**

1. Write a program to display a the multiplication table of the numbers from 1 to ten upto ten terms.
2. Write a program to display the following output:
 

|       |          |          |       |
|-------|----------|----------|-------|
| a) 1  | b) 12345 | c) 54321 | d) 1  |
| 12    | 1234     | 5432     | 11    |
| 123   | 123      | 543      | 111   |
| 1234  | 12       | 54       | 1111  |
| 12345 | 1        | 5        | 11111 |
3. Write a program to ask the Marks in five subjects each of 10 students and display the total percentage, result and division.

Some of the examples of nested loop are discussed in the Array and string sections.

## Arrays

Array is an ordered collection of identical elements which is represented by a single variable name. Sometimes it is necessary to store a large number of similar types of data in different variables. In such case array may be used to avoid the burden of using large number of variables.

For example:

If we have to read the heights of 50 persons and store in different variables for future use then in such case we can simply do that using an integer type of array 'A[50]'. The array A[50] can store up to fifty data in by separating 50 memory locations, from A[0] to A[49].

### Declaration of Arrays

The arrays can be declared by the same process as that of the variables except that the size of the array must be given in the declaration phase.

Syntax:

Datatype variable\_name [size] ;

Example:

int Marks[5];

The above statement declares an array 'Marks', which reserves 5 memory locations for storing integer type of data. The names of the memory locations are given as **Marks[0]**, **Marks[1]**, **Marks[2]**, **Marks[3]** and **Marks[4]**. The number within the square brackets is known as index or subscript. A one dimensional array contains a single subscript while a two dimensional array contains two subscripts.

### Assigning values to arrays

The assignment of values to the arrays is also somewhat similar to that of variables. If an array A[10] is declared as an integer type then we can assign value to it by the following methods:

a) A [0] = 35;  
A [1] = 56;  
:  
:  
A [8] = 90;  
A [9] = 45;

b) c) A[10]={23,34,54,65,67,98,78,48,41,24}  
Here, the value 23 is assigned to A[0], 34 to A[1], 54 to A[2] and so on.

#### Direct input

```
for(i=0;i<=9;i++)
{
scanf("%d",&A[i]);
}
```

Of the above three examples, in the first one we have simply assigned values to the array variables as in simple variables using assignment operator. In the second one we have directly assigned the values to all the array elements. And in the third one we have read the value through the keyboard and stored in the array elements using scanf() statement inside loop.

```
#include <conio.h>
#include <stdio.h>

void main()
{
    int A[10], i;
    printf("Enter the marks\n");
    for (i=0; i<=9; i++)
    {
        scanf("%d", &A[i]);
    }
    for (i=1; i<=9; i++)
    {
        printf("%d ", A[i]);
    }
    getch();
}
```

```
Enter the marks
8
90
56
34
2
12
23
45
78
90
90 56 34 2 12 23 45 78 9
```

Fig: The program alongside is designed to accept ten data from the keyboard and store in array variables and display them.

The screenshot shows a C program in a text editor and its execution in a console window. The program calculates the total marks, percentage, and division based on marks entered for eight subjects.

```

#include "conio.h"
#include "stdio.h"

void main()
{
    int A[10], i, Total=0; float per;
    printf("Enter the marks in eight sub");
    for (i=0; i<=7; i++)
    {
        scanf("%d", &A[i]); Total=Total+A[i];
    }
    printf("Total=%d\n", Total);
    per=float(Total)/8;
    printf("Percent=%.2f\n", per);
    for (i=0; i<=7; i++)
    {
        if (A[i]<=32)
        {
            printf("Result=Fail");
            printf("\nDivision:*****");
            goto Label;
        }
        printf("You're Pass\n");
        if (per>=80)
            printf("Division:Dist");
        else if (per>=60)
            printf("Division:First");
        else if (per>=45)
            printf("Division:Second");
        else
            printf("Division:Third");
    }
    Label:
    getch();
}

```

Execution Output (Top Window):

```

Enter the marks in eight subjects
78
90
87
76
56
45
99
89
Total=620
Percent=77.50
You're Pass
Division:First

```

Execution Output (Bottom Window):

```

Enter the marks in eight subjects
98
90
78
99
98
99
98
31
Total=691
Percent=86.38
Result=Fail
Division:*****

```

Fig: A program to calculate and display Total, percentage, result and division if Marks in eight subjects are supplied by the user.

The screenshot shows a C program in a text editor and its execution in a console window. The program sorts ten numbers entered by the user in descending order.

```

#include <stdio.h>
#include <conio.h>

void main()
{
    int i, j, x, a[10];
    printf("Enter ten numbers\n");
    for (i=0; i<=9; i++)
        scanf("%d", &a[i]);
    for (i=0; i<=9; i++)
        for (j=0; j<=8; j++)
        {
            if (a[j+1]>a[j])
            {
                x=a[j];
                a[j]=a[j+1];
                a[j+1]=x;
            }
        }
    printf("\nThe numbers in descendent order are:");
    for (i=0; i<=9; i++)
        printf("%d ", a[i]);
    getch();
}

```

Execution Output:

```

Enter ten numbers
12
32
45
78
65
34
21
67
30
45
The numbers in descending orders are:
78 67 65 45 45 34 32 30 21 12

```

Fig: A program to sort ten different numbers provided by the users in descending order.

## Two dimensional Array

The arrays that contain two subscripts is known as two dimensional arrays. It can store the data as in rows and columns of a matrix. For example: The two dimensional array  $A[2][3]$  can hold  $2 \times 3 = 6$  number of data in its storage locations given by

$A[0][0]$   $A[0][1]$   $A[0][2]$   
 $A[1][0]$   $A[1][1]$   $A[1][2]$

A two dimensional array can be declares as follows:

Syntax:

datatype variable [index1] [index2];

Where index1 and index2 are the integer value that represent the number of rows and columns. The total size of data that the array can hold is [index1×index2].

Example:

int Marks[20][20];

## Assigning values to two dimensional array

If an array is declared  $A[2][3]$  We can assign the values to the array elements by the following methods.

a)  $A[0][0] = 35;$   
 $A[0][1] = 56;$   
 $A[0][2] = 34;$   
 $A[1][0] = 55;$   
 $A[1][1] = 76;$   
 $A[1][2] = 57;$

b)  $A[2][3] = \{23, 34, 54, 65, 67, 98\}$   
 Here, the value 23 is assigned to  $A[0][0]$ , 34 to  $A[0][1]$ , 54 to  $A[0][2]$  and so on.

### Direct input

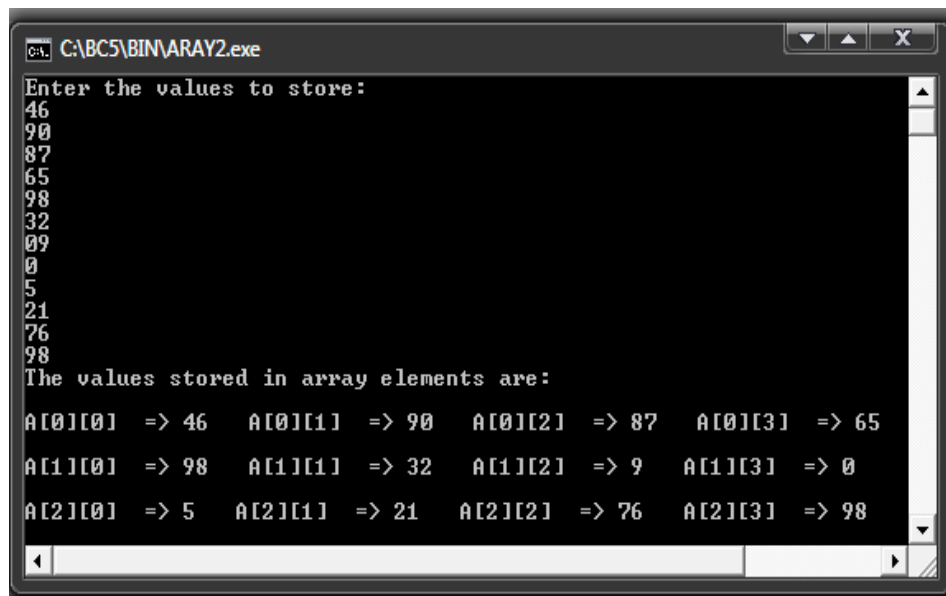
```
for(i=0;i<=9;i++)
{
    for(j=0;j<=3;j++)
        scanf("%d",&A[i][j]);
}
```

In the above three examples, in the first one individual array element is assigned a value using assignment operator. In the second one all the value are fed at once by the programmer and in the third one, the statement reads six different data from the keyboard and store in the individual array element.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,A[3][4];
    clrscr();
    printf("Enter the values to store:\\n");
    for(i=0;i<3;i++)
        for(j=0;j<4;j++)
            scanf("%d",&A[i][j]);
    printf("The values stored in array elements are:\\n\\n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<4;j++)
            printf("A[%d][%d] => %d  ",i,j,A[i][j]);
        printf("\\n\\n");
    }
    getch();
}
```

Output on the  
next page

Fig: A program to store different data in Two dimensional array and display it along with the name of storage location.



```

C:\BC5\BIN\ARRAY2.exe
Enter the values to store:
46
90
87
65
98
32
09
0
5
21
76
98
The values stored in array elements are:
A[0][0] => 46  A[0][1] => 90  A[0][2] => 87  A[0][3] => 65
A[1][0] => 98  A[1][1] => 32  A[1][2] => 9  A[1][3] => 0
A[2][0] => 5   A[2][1] => 21  A[2][2] => 76  A[2][3] => 98

```

Fig: Output of the above program.

```

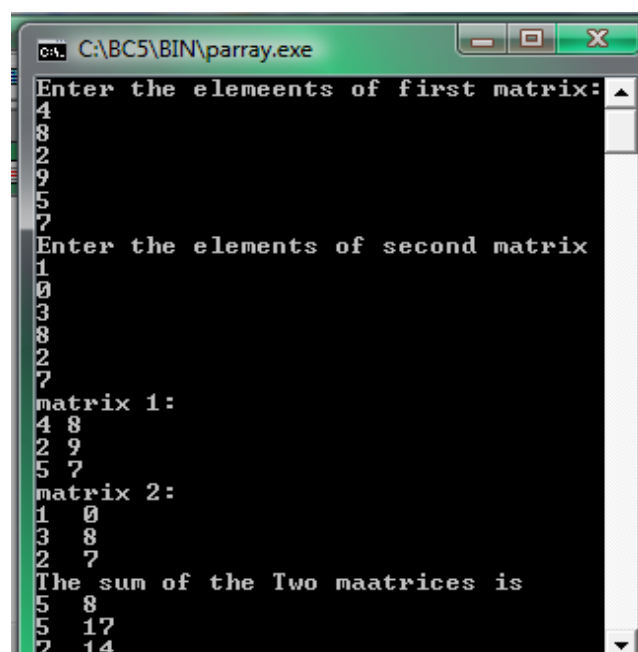
#include<stdio.h>
#include<conio.h>
void main()
{
    int i, j, A[3][2], B[3][2], Sum[3][2];
    clrscr();
    printf("Enter the elements of first matrix:\n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<2;j++)
        {
            scanf("%d",&A[i][j]);
        }
    }
    printf("Enter the elements of second matrix\n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<2;j++)
        {
            scanf("%d",&B[i][j]);
        }
    }
    for(i=0;i<3;i++)
    {
        for(j=0;j<2;j++)
        {
            Sum[i][j]=A[i][j]+B[i][j];
        }
    }
    printf("matrix 1:\n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<2;j++)
        {
            printf("%d ",A[i][j]);
        }
        printf("\n");
    }
    printf("matrix 2:\n");
    for(i=0;i<3;i++)
    {

```

```

        for(j=0;j<2;j++)
        {
            printf("%d ",B[i][j]);
        }
        printf("\n");
    }
    printf("The sum of the Two matrices is");
    for(i=0;i<3;i++)
    {
        for(j=0;j<2;j++)
        {
            printf("%d ",Sum[i][j]);
        }
        printf("\n");
    }
    getch();
}

```



```

C:\BC5\BIN\pararray.exe
Enter the elements of first matrix:
4
8
2
9
5
7
Enter the elements of second matrix
1
0
3
8
2
7
matrix 1:
4 8
2 9
5 7
matrix 2:
1 0
3 8
2 7
The sum of the Two matrices is
5 8
5 17
7 14

```

Fig: A program to add two matrices provided by

**Array of string**

We discussed a lot about the numeric arrays in above chapters. In C we can also assign the character arrays. We cannot store a string (large or small) in the variables. For storing string we require array of characters. Each array element can hold a single character. We have to assign a character array 'A[10]' to store a string 'Kathmandu'. We can consider the size of array as the length of string in case of string arrays example:

char a[10]="Kathmandu" is stored as:

a[0]='K'      a[1]='a'      a[2]='t'      a[3]='h' ..... a[8]='u'      a[9]='\0'

The string is terminated by the blank character '\0' which is automatically added at the end of every string.

**Assigning values to string arrays**

Assigning values to string array is a bit different than that of the numeric array. Each element of A numeric array can hold the number of more than one digit, as supported by the data type, the elements of string array can hold just a single character rather than a word or a sentence. Blank spaces are also termed as a character.

Example:

```
int A[10];
A[0]='A';
A[1]='P';
A[2]='P';
A[3]='L';
A[4]='E';
printf("%s",A);
```

The above program prints 'apple'. and is stored as

|   |   |   |   |   |    |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|
| A | P | P | L | E | \0 |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|

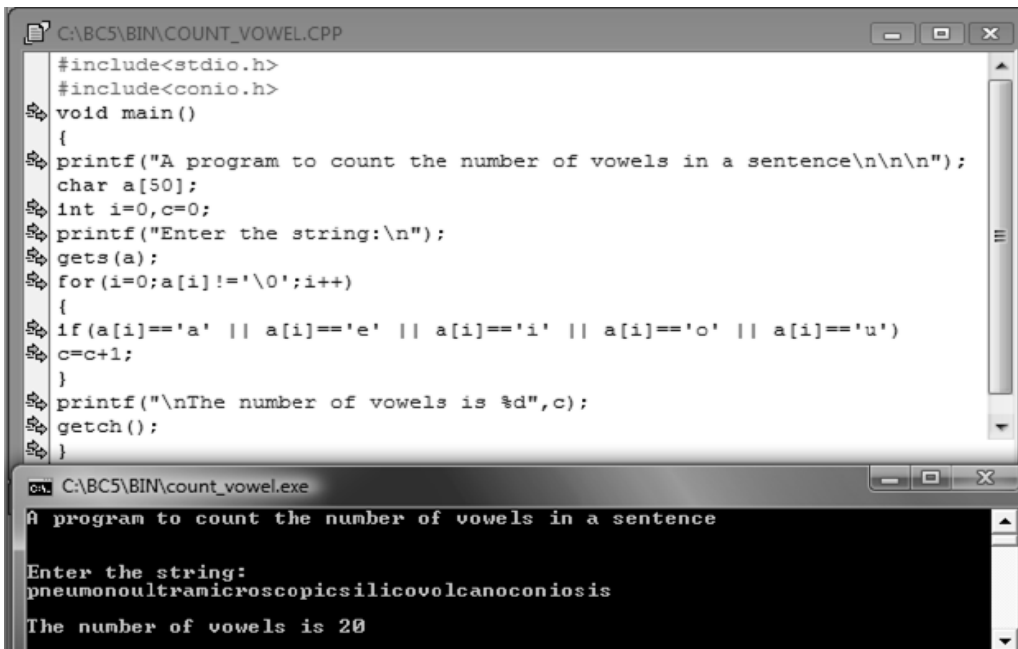
```
#include<stdio.h>
#include<conio.h>
void main()
{
    char a[10]="Kathmandu";
    printf("%s",a);
    getch();
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char a[10];
    int i;
    printf("Enter the characters\n");
    scanf("%s",&a);
    printf("\n\nThe string is \n%s",a);
    getch();
}
```

The program aside printed 'Script', which is not the expected result. It is so because the scanf() function terminates by the first blank space. After reading Script it encountered a blank space so it did not read the remaining words.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char a[50];
    printf("Enter string\n");
    gets(a);
    printf("\n\nThe string is\n");
    printf("%s",a);
    getch();
}
```

The above program is corrected by replacing scanf() function with gets(). Gets() function reads the string until the user presses 'Enter'.



The screenshot shows two windows. The top window is a code editor titled 'C:\BC5\BIN\COUNT\_VOWEL.CPP' containing the following C code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
printf("A program to count the number of vowels in a sentence\n\n\n");
char a[50];
int i=0,c=0;
printf("Enter the string:\n");
gets(a);
for(i=0;a[i]!='\0';i++)
{
if(a[i]=='a' || a[i]=='e' || a[i]=='i' || a[i]=='o' || a[i]=='u')
c=c+1;
}
printf("\n\nThe number of vowels is %d",c);
getch();
}
```

The bottom window is a command prompt titled 'C:\BC5\BIN\count\_vowel.exe' showing the program's execution:

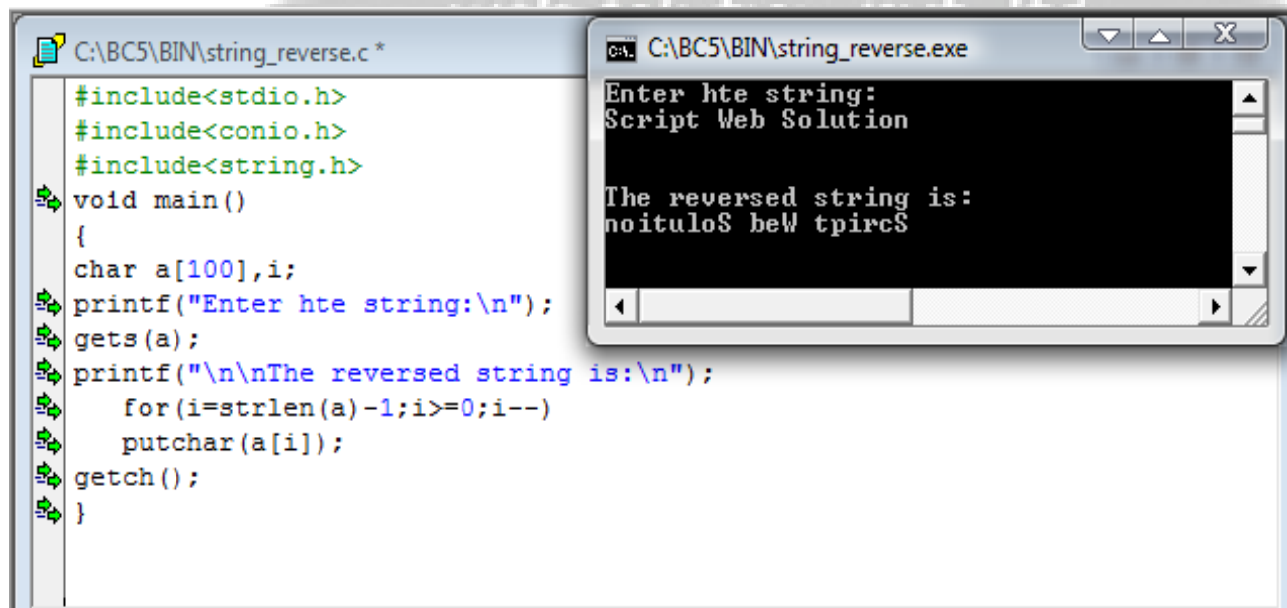
```
A program to count the number of vowels in a sentence

Enter the string:
pneumonoultramicroscopicsilicovolcanoconiosis

The number of vowels is 20
```

Fig: The program aside counts the number of vowels in the string entered by the users.

### A program to reverse a string:



The screenshot shows two windows. The top window is a code editor titled 'C:\BC5\BIN\string\_reverse.c \*' containing the following C code:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
char a[100],i;
printf("Enter hte string:\n");
gets(a);
printf("\n\nThe reversed string is:\n");
for(i=strlen(a)-1;i>=0;i--)
putchar(a[i]);
getch();
}
```

The bottom window is a command prompt titled 'C:\BC5\BIN\string\_reverse.exe' showing the program's execution:

```
Enter hte string:
Script Web Solution

The reversed string is:
noituloS beW tpircS
```

## Some string handling functions

In C Library different string handling functions are stored in the header file **<string.h>** in order to manipulate the strings stored in different variables.

### **Strcpy()**

The strcpy() function is used to store the string from one variable to another. It can also be used to store a string constant into certain character variables. Strings, in C can't directly be assigned to the variables using assignment operator '=', so for that purpose we can use the strcpy() function.

Syntax:

**Strcpy( destination , source );**

The **destination** part must always contain a string variable of size enough to hold the data of the **source** variable but the **source** part may contain 'the string variable having certain value' or 'a string constant itself'.

Example:

**Strcpy( name , "Rameshwor Rana ")** copies\stores the name **Rameshwor Rana** in the variable **name**(if the size of character variable **name** is large enough to store the given string **Rameshwor Rana.**)

**Strcpy( add , address);** If **add** and **address** are two different string variables of certain size then the given example copies the string value of **address** to **add**. Here the size of add must be at least equal to that of address.

### **Strcmp()**

This function is used to compare two different strings and return an integer value. The value it returns is the numerical difference between the ASCII value of first non matching character in the first string and that of the second one. Two string are said to be equal if the function returns the value zero(0), else it is not equal. It is very useful for sorting the words or string.

Syntax:

**Strcmp(str\_variable1,str\_variable2);**

Example:

**Strcmp("Ram","Ranjan") ;** returns the integer value **(-1)** which is the difference between the ascii value of 'm'(of ram) to 'n'(of Ranjan).

### **Strrev()**

This function is used to reverse a string. This function returns the reverse of the string stored in certain variable.

Syntax:

**Strrev( variable\_name );**

Example:

**A[10]="Kathmandu";**

**Printf("%s",strrev(a[10]));** prints the text **udnamhtaK on the monitor.**

### **Strcat()**

This function is used to combine (concatenate) two different strings and store the combined string in single variable.

**Syntax:****Strcat(var\_1,var\_2);****Example:**

**Strcat(a,b);** combines the string **b** with **a** and stores in variable **a**. The size of **a** must be large enough to hold the string of both **a** and **b**.

**Strlen()**

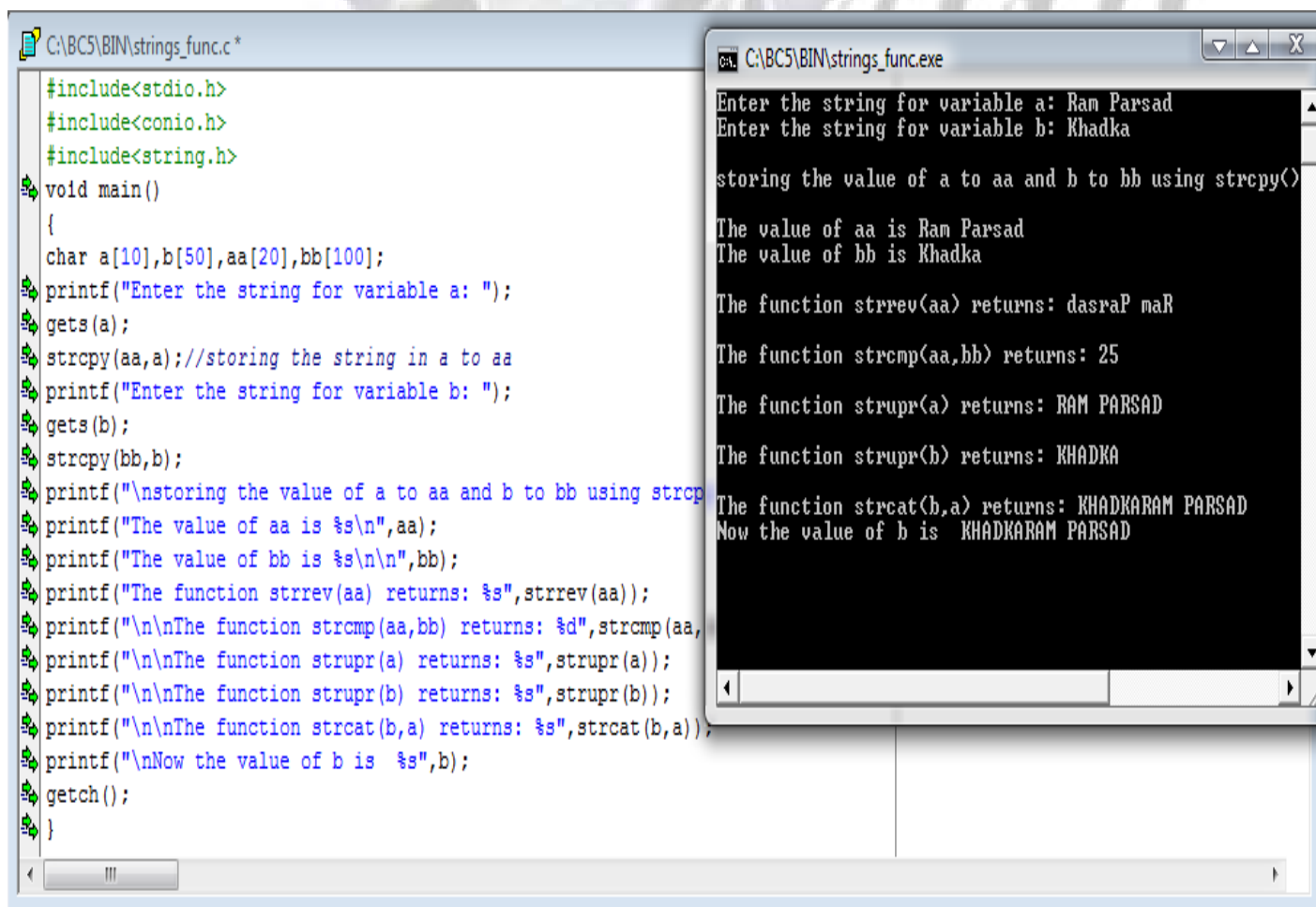
This function is used to count the length of given string.

**Syntax:****Strlen(string\string\_variable)****Example:**

**A[10]="Kath"**

**Strlen(A)** returns the integer value 4 which is the length of A.

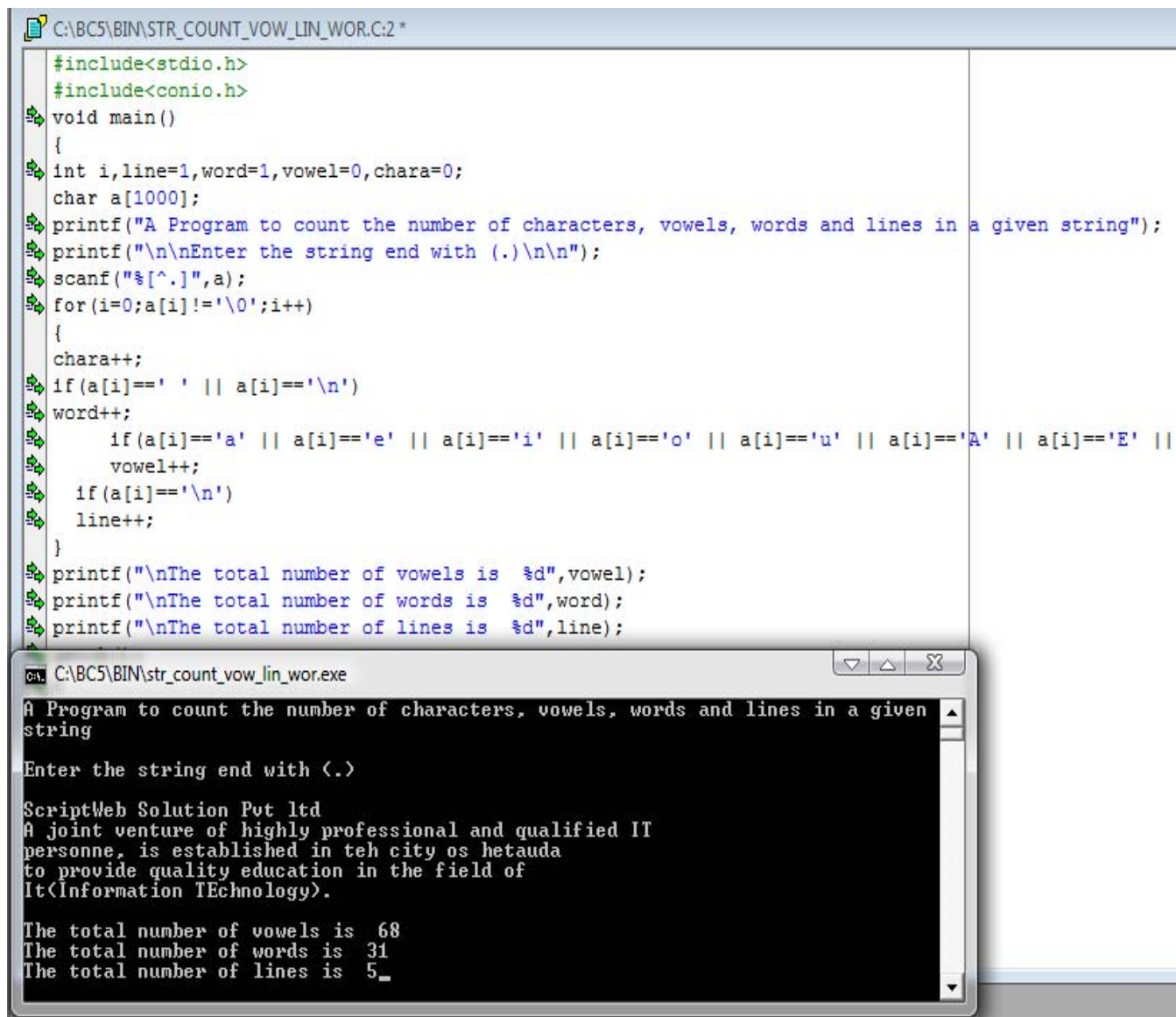
**Note:** Other functions like **strlwr()** to convert the string into lower case **strupr()** to convert string to uppercase are also stored in the **string.h** header file.



```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char a[10],b[50],aa[20],bb[100];
    printf("Enter the string for variable a: ");
    gets(a);
    strcpy(aa,a);//storing the string in a to aa
    printf("Enter the string for variable b: ");
    gets(b);
    strcpy(bb,b);
    printf("\nstoring the value of a to aa and b to bb using strcpy()");
    printf("The value of aa is %s\n",aa);
    printf("The value of bb is %s\n",bb);
    printf("The function strrev(aa) returns: %s",strrev(aa));
    printf("\n\nThe function strcmp(aa,bb) returns: %d",strcmp(aa,bb));
    printf("\n\nThe functionstrupr(a) returns: %s",strupr(a));
    printf("\n\nThe functionstrupr(b) returns: %s",strupr(b));
    printf("\n\nThe function strcat(b,a) returns: %s",strcat(b,a));
    printf("\nNow the value of b is %s",b);
    getch();
}
```

Enter the string for variable a: Ram Parsad  
Enter the string for variable b: Khadka  
storing the value of a to aa and b to bb using strcpy()  
The value of aa is Ram Parsad  
The value of bb is Khadka  
The function strrev(aa) returns: dasraP naR  
The function strcmp(aa,bb) returns: 25  
The functionstrupr(a) returns: RAM PARSAD  
The functionstrupr(b) returns: KHADKA  
The function strcat(b,a) returns: KHADKARAM PARSAD  
Now the value of b is KHADKARAM PARSAD

Fig: The demonstration of String handling functions.



```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,line=1,word=1,vowel=0,chara=0;
    char a[1000];
    printf("A Program to count the number of characters, vowels, words and lines in a given string");
    printf("\n\nEnter the string end with (.)\n\n");
    scanf("%[^.]",a);
    for(i=0;a[i]!='\0';i++)
    {
        chara++;
        if(a[i]==' ' || a[i]=='\n')
            word++;
        if(a[i]=='a' || a[i]=='e' || a[i]=='i' || a[i]=='o' || a[i]=='u' || a[i]=='A' || a[i]=='E' ||
            vowel++;
        if(a[i]=='\n')
            line++;
    }
    printf("\nThe total number of vowels is  %d",vowel);
    printf("\nThe total number of words is  %d",word);
    printf("\nThe total number of lines is  %d",line);
}
```

CA\ABC5\BIN\str\_count\_vow\_lin\_wor.exe

A Program to count the number of characters, vowels, words and lines in a given string

Enter the string end with (.)

ScriptWeb Solution Pvt ltd  
A joint venture of highly professional and qualified IT  
personne, is established in teh city os hetauda  
to provide quality education in the field of  
It<Information TEchnology>.

The total number of vowels is 68  
The total number of words is 31  
The total number of lines is 5\_

In the above program we can detect the use of new concept. i.e. the use of the `%[^.]` format specifier in place of the other like `%d`, `%s`, `%f` etc. The `%[^.]` tells the compiler to read the text until the user presses period (.). The program is terminated as soon as it encounters a period(.) in the string entered by the user.

Similarly `%[^_]` terminates the string with the encounter with the underscore (`_`) sign.

**Exercises:**

- i) Write a program to store different values in a one dimensional array and print the array elements using two different loops.
- ii) Write a program to store n numbers in an array and:
  - a. Print all the odd numbers only.
  - b. Print the prime numbers only.
  - c. Print the numbers divisible by 7 and not by 8.
  - d. Print the numbers which are palindrome.
  - e. Print the numbers between 30 and 70.
  - f. Calculate the sum and average of the numbers entered.
  - g. Compute the greatest and smallest number and display it on the monitor.
  - h. Sort the numbers and display them in:
    - i. Ascending order
    - ii. Descending order
  - i. Calculate and display the factorial of each numbers.

**Strings**

- iii) Write a program to accept any length of string from the users and display it.
- iv) Write a program to read a multiple line text(string) from the user and display it on the monitor.
- v) Write a program to reverse a given set of string without using `strrev()` function.
- vi) Show the proper use of different string handling functions like:
  - i. `strcpy()`
  - ii. `strcat()`
  - iii. `strupr()`
  - iv. `strrev()`
  - v. `strlen()`
  - vi. `strlwr()`
- vii) Write a program to ask a string from the user and count:
  - i. The number of characters with blank space.
  - ii. The number of characters without blank space.
  - iii. The number of vowels & consonants.
  - iv. The number of words.
  - v. The number of lines.
- viii) Write a program in a quiz style that must ask a question and check the answer in words.
- ix) Write a program to check whether a string entered by the user is palindrome or not.

**Multi Dimensional Array**

- x) Write a program to enter a set of data in a two dimensional array and display it in the form of a matrix.
- xi) Write a program to enter two different matrices and display the sum of the matrices.(Matrix addition)
- xii) Subtract two different matrices.
- xiii) Write a program to store the name of ten persons and:
  - a. Display the name that starts with a given letter.
  - b. Display the name with the given length.
  - c. Sort the names and display on the monitor.

## Functions In C

Every C program is a collection of functions. We have already discussed about different functions like `Printf()`, `scanf()`, `gets()`, `puts()`, `main()` etc. in the previous topics. The functions in C are classified into two main categories. They are:

I) Library functions:

The library functions like `printf()`, `getch()`, `clrscr()`, `pow()` etc. are primarily coded by the manufacturer and stored in the C library. All such functions are given specific task. For example the `printf()` function is used to display different text on the monitor while the `clrscr()` function to clear the output screen. Similarly the `pow()` function, to calculate the power of different numbers. The library functions are also called built in function as they are built in with the C compiler, and the user mustn't bother to code them.

II) User defined functions:

The user defined functions have no meaning in themselves. They need to be defined by the users at the time of programming. The `main()` function also an example of the user defined function that normally don't return any value so sometimes it is declared void.

**A function is a group of statements that form a single logical unit and perform some specific task.**

The large programs are divided into many small functions and joined together that help for decreasing the size of a program efficiently. Functions make program much readable and understandable. Function declared at one place can be called from any place within the program.

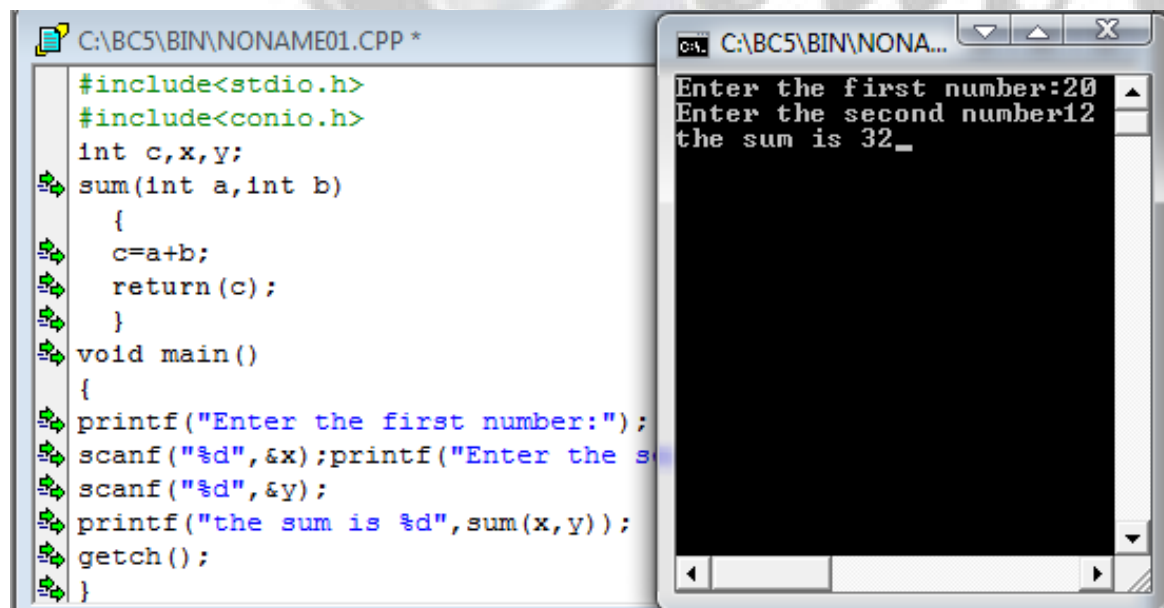


Fig: A simple program that asks two different numbers and displays their sum as output using Sum function.

### Basic structure of a function

Data-type Function\_name(parameter list)

Declarations o arguments;

```
{
    Local variable declaration;
    Statement block;
    return(expression);
}
```

The function name is an identifier defined by the users. Parameter list are the arguments of different data-types which are processed through the function. Statement block contains the set of statement that is executed when the function is called

Basically, every function return a value to the calling function. However some function like main(), that we have already used before, doesn't return any value.

### **Function Calling**

The main executable part of any C programs is the main() function. So, in order to bring other functions in action we must call them through the main function.

Syntax for calling function:

Main()

```
{
:
:
function name();
:
}
```

In the above program the statement:  
'printf("The sum is %d",sum(x,y))'  
Is used to call the sum function.

User defined functions can further be categorized into Three groups:

### **Function that has no arguments and no return value**

Such functions that neither receive any data(value) from the calling function nor do return any value to the calling function after execution fall under this group. This type of function is static which is very rarely used.

For example:

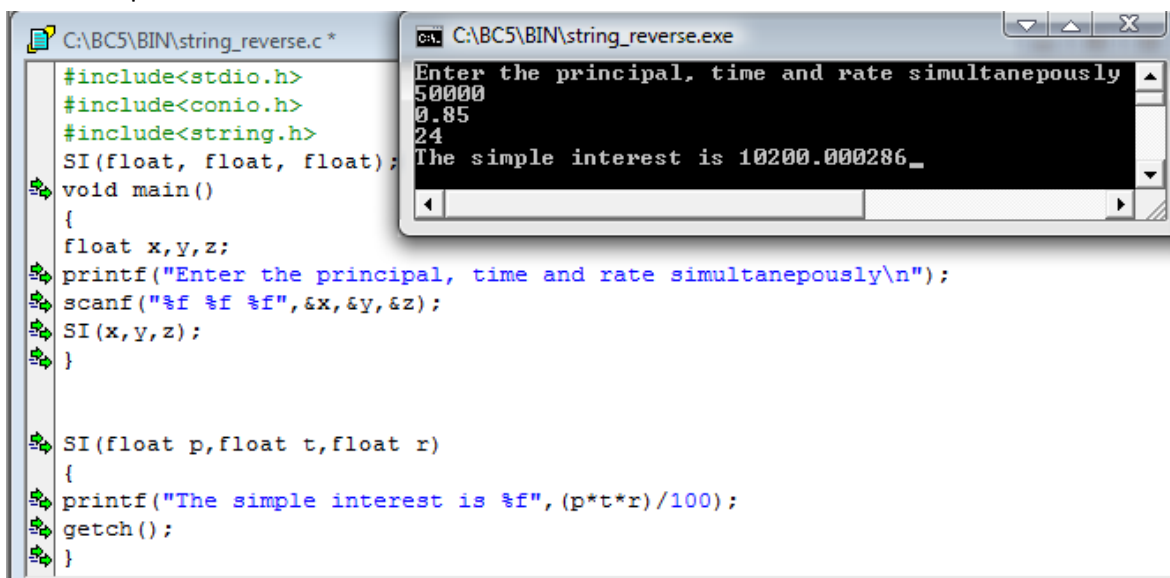
```
print_name()
{
printf("Script Web Solution Pvt. Ltd.")
Return;
}
```

The above function is used just to display the text inside the function.

### **Function that has arguments but no return value:**

Such function that receives data from the calling function but doesn't return any value to the calling function falls under this category. It, after receiving value(through arguments) processes the arguments and execute them rather returning the value to the calling function.

For example:



```
C:\BC5\BIN\string_reverse.c *
#include<stdio.h>
#include<conio.h>
#include<string.h>
SI(float, float, float);
void main()
{
float x,y,z;
printf("Enter the principal, time and rate simultaneously\n");
scanf("%f %f %f",&x,&y,&z);
SI(x,y,z);
}

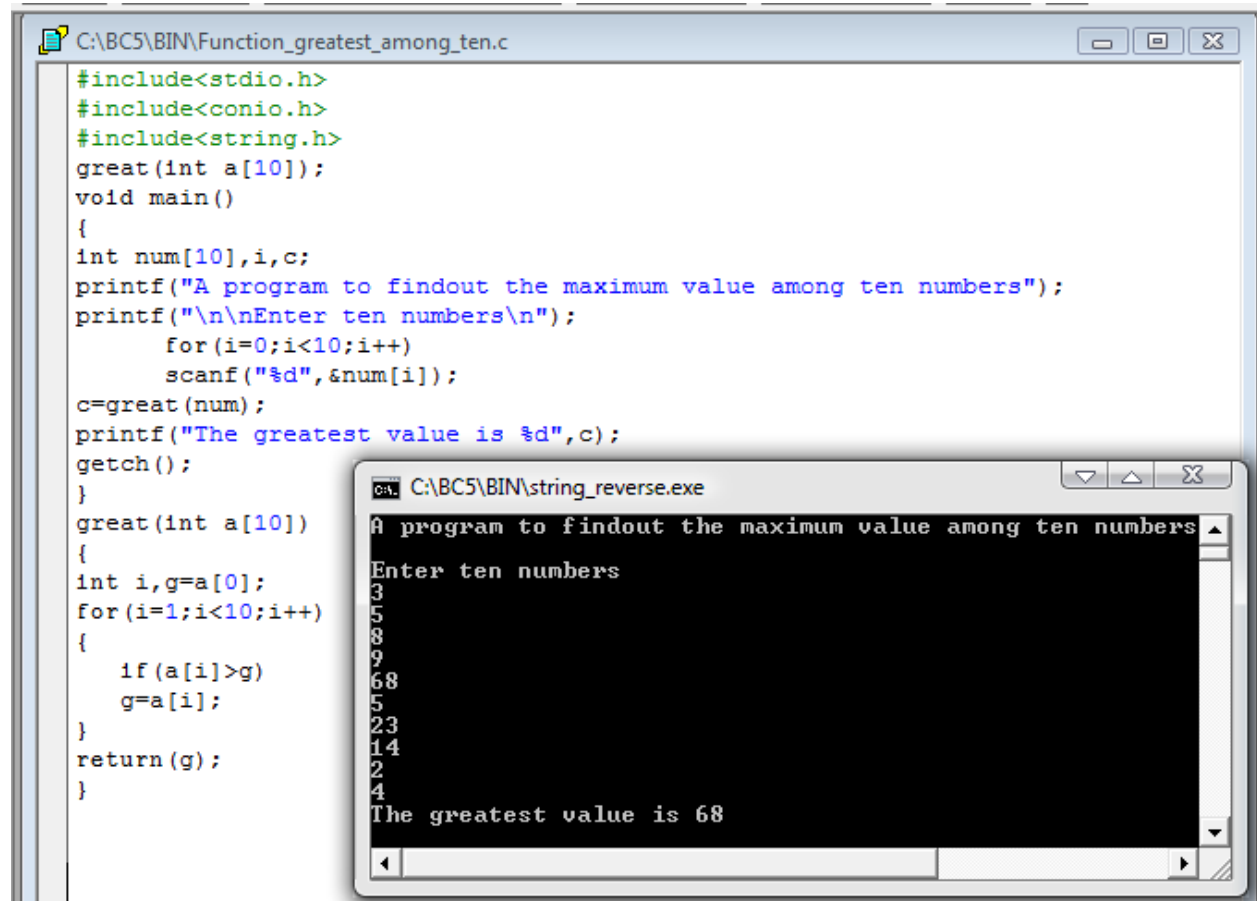
SI(float p,float t,float r)
{
printf("The simple interest is %f",(p*t*r)/100);
getch();
}
```

```
C:\BC5\BIN\string_reverse.exe
Enter the principal, time and rate simultaneously
50000
0.85
24
The simple interest is 10200.000286_
```

Fig: A program to print the simple interest using function if p,t,r is supplied by the user.

**Function that has arguments and returns a value**

This category cover the function that receive certain set of data from the calling function, process them and return certain value(output) to the calling function. It is the most useful type of function.



The image shows a screenshot of a C program and its execution. The main window displays the source code for a program named 'Function\_greatest\_among\_ten.c'. The code includes headers for stdio, conio, and string, and defines a 'greatest' function that takes an array of 10 integers and returns the maximum value. The 'main' function prompts the user to enter ten numbers and calls the 'greatest' function. An inset window shows the program's execution, where the user has entered ten numbers (3, 5, 8, 9, 68, 5, 23, 14, 2, 4) and the program has outputted 'The greatest value is 68'.

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
greatest(int a[10]);
void main()
{
    int num[10],i,c;
    printf("A program to findout the maximum value among ten numbers");
    printf("\n\nEnter ten numbers\n");
    for(i=0;i<10;i++)
        scanf("%d",&num[i]);
    c=greatest(num);
    printf("The greatest value is %d",c);
    getch();
}
greatest(int a[10])
{
    int i,g=a[0];
    for(i=1;i<10;i++)
    {
        if(a[i]>g)
            g=a[i];
    }
    return(g);
}
```

A program to findout the maximum value among ten numbers  
Enter ten numbers  
3  
5  
8  
9  
68  
5  
23  
14  
2  
4  
The greatest value is 68

**Exercises:**

- i) Write a program using function to print any message on the screen.
- ii) Write a program using a function that has arguments and returns certain value to find out the sum and product of two different numbers entered by the user.
- iii) Write a program to supply three different numbers to a function "greatest()" and find out the greatest among the three numbers.
- iv) Write a program using function to sort the numbers in an array of n numbers and print it on the monitor.
- v) Write a program using recursive function to find the sum of numbers from 1 to 100.